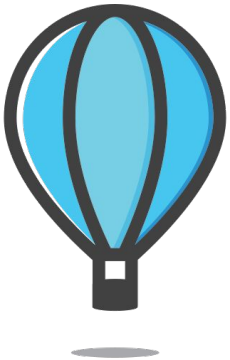
Nav2 - what is ?

- An open source framework for mobile robot navigation designed for ROS 2, “the go-to industry-standard system”
- Professionally-supported successor of the ROS Navigation Stack, deploying the same kinds of technology powering autonomous vehicles
- Allows for mobile robots to navigate through complex environments to complete user-defined application tasks with nearly any class of robot kinematics
- Provides perception, planning, control, localization, obstacle avoidance, visualization and more



About Nav2: https://docs.nav2.org/

N A V 2

Nav2 - how does it work ?

- Nav2 uses **behavior trees** (BTs) to create customized and intelligent navigation behavior via orchestrating many independent modular **servers**
    - Behavior tree is a mathematical model of plan execution, opposed to a finite state machine
    - BTs are becoming increasingly common in complex robotics tasks, as explained in this book
- A task server can be used to compute a path, control effort, behavior, or any other navigation related task
- These separate servers communicate with the behavior tree (BT) over a ROS interface such as an **action server** or service

Nav2 - how do



- Nav2 uses **behav** nd intelligent navigation behav modular **servers**
  - Behavior tr cution, opposed to a finite state
  - BTs are bec x robotics tasks, as explained i
- A task server can ort, behavior, or any other navigation
- These separate s ree (BT) over a ROS interface such as an **action server** or service

“nobody uses actions in ROS”



“we won’t explain actions in this course”

“I’ve never seen an action being used”

“you’ll only use topics and, sometimes, services”

Well, it’s time…

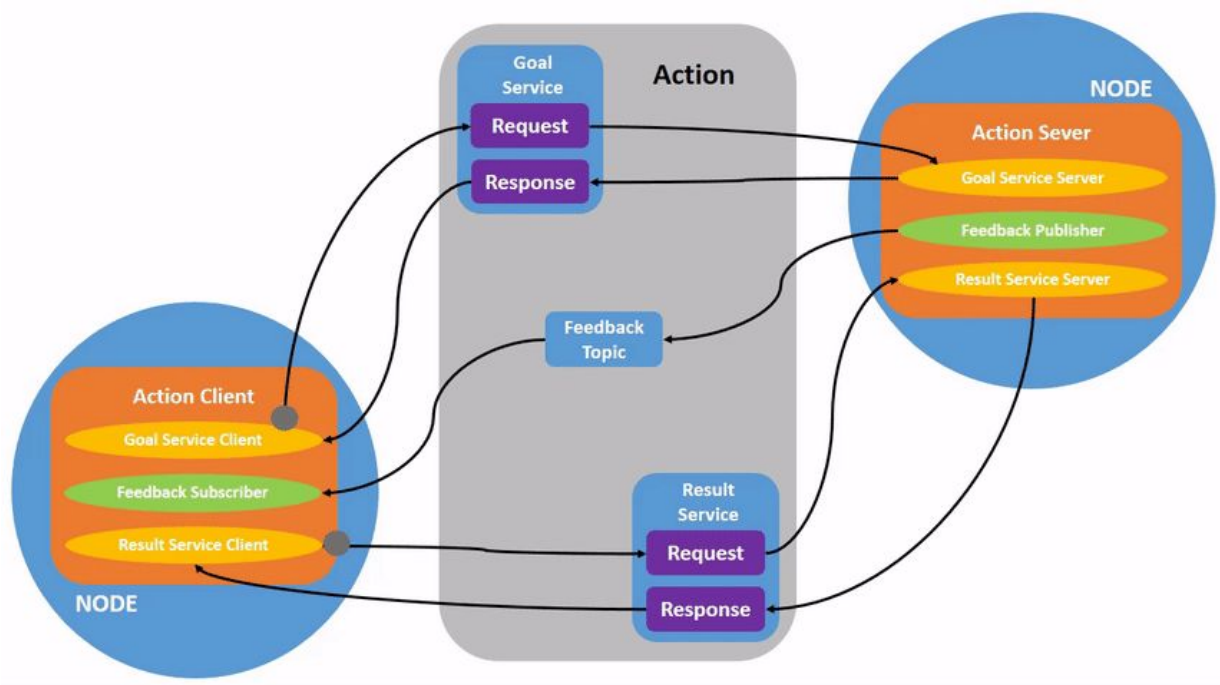
Action - what is an action ?

- Enable a client to initiate a **long-running task** and receive **periodic feedback**
- Actions consist of three parts: a **request**, a **result**, and **feedback**
- The action *server* receives a **request** from an action *client* (a service) and periodically sends **feedback** (over a topic) until the action is complete, whereupon it sends a **result** (a service response)
- Actions are implemented, at a lower-level, in terms of **services** and **topics**
    - These services and topics are hidden by default, and passing `--include-hidden-topics` to `ros2 topic` and `--include-hidden-services` to `ros2 service` provides access to them

About actions: https://docs.ros.org/en/jazzy/Concepts/Basic/About-Actions.html

Action - what is an action ?

- Enable a client to initiate a **long-running task** and receive **periodic feedback**

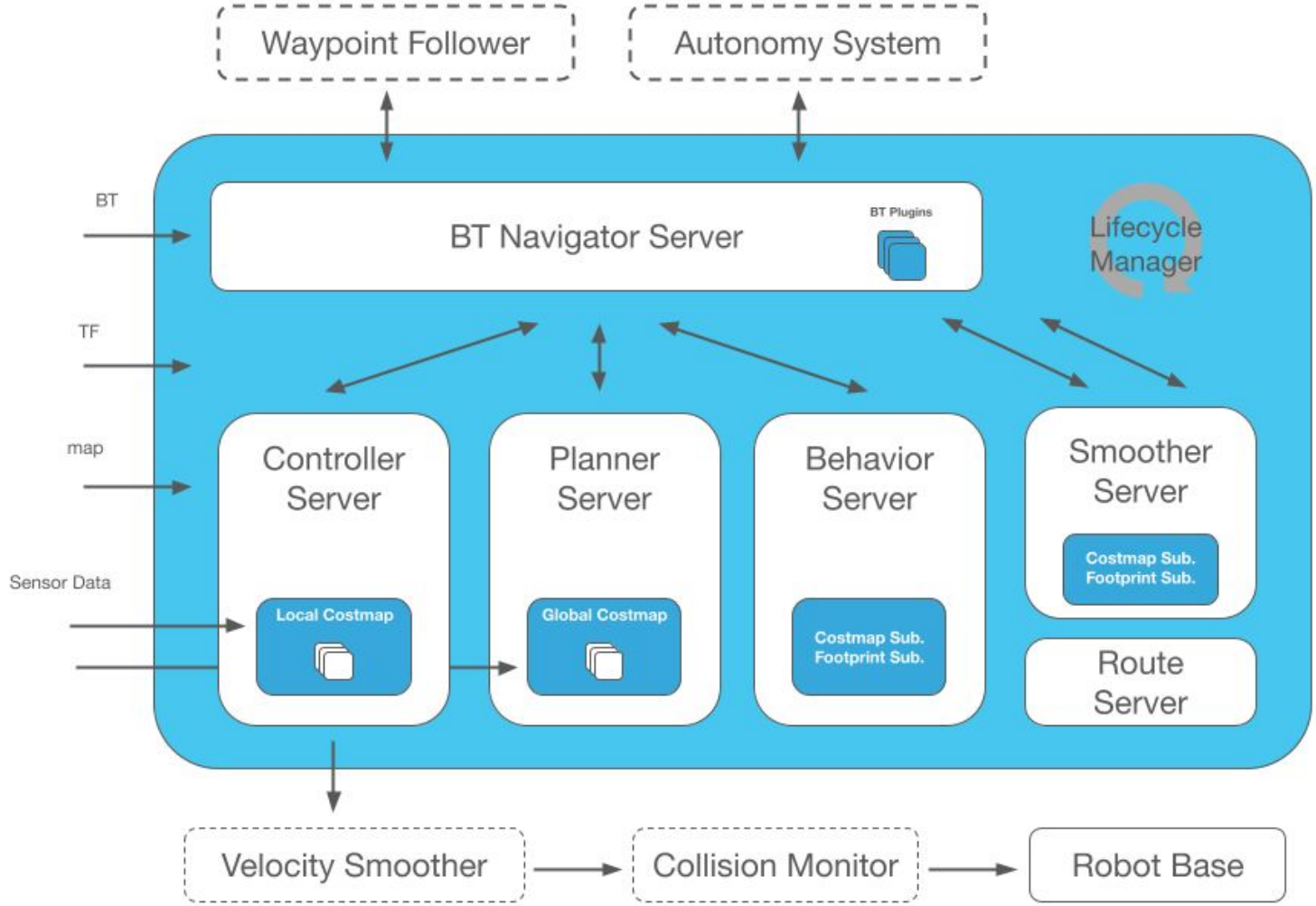


https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html

Action server - uses in Nav2

- As in ROS, action servers are a common way to control long running tasks like navigation
- A client will request some task to be completed, and this task may take a long time. An example would be moving the shovel up from a bulldozer or ask a robot to travel 10 meters to the right
- Examples of the use of action servers are the **navigation servers**, mainly the planner, behavior, smoother, route and controller servers
- The diagram in the next page will give you a good first-look at the main structure of Nav2

About Nav2 concepts: https://docs.nav2.org/concepts/index.html

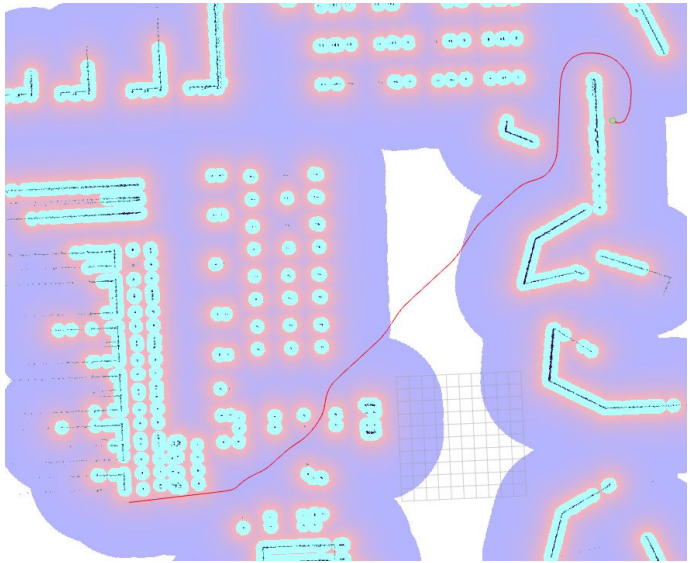

Waypoint Follower
Autonomy System
BT
BT Navigator Server
BT Plugins
Lifecycle Manager
TF
map
Sensor Data
Controller Server
Local Costmap
Planner Server
Global Costmap
Behavior Server
Costmap Sub.
Footprint Sub.
Smoother Server
Costmap Sub.
Footprint Sub.
Route Server
Velocity Smoother
Collision Monitor
Robot Base

Planner server

- The task of a planner is to compute a **path** to complete some objective function
- The path can also be known as a **route**, depending on the nomenclature and algorithm selected
- Examples are computing a plan to a **goal** (*e.g.* from current position to a goal) or **complete coverage** (*e.g.* plan to cover all free space)
- Planners can be written to:
  - Compute shortest path
  - Compute complete coverage path
  - Compute paths along sparse or predefined routes

Planner server

- The planner will have access to a **global environmental representation** and **sensor data** buffered into it
- The general task in Nav2 for the planner is to compute a **valid**, and **potentially optimal**, path from the current pose to a goal pose

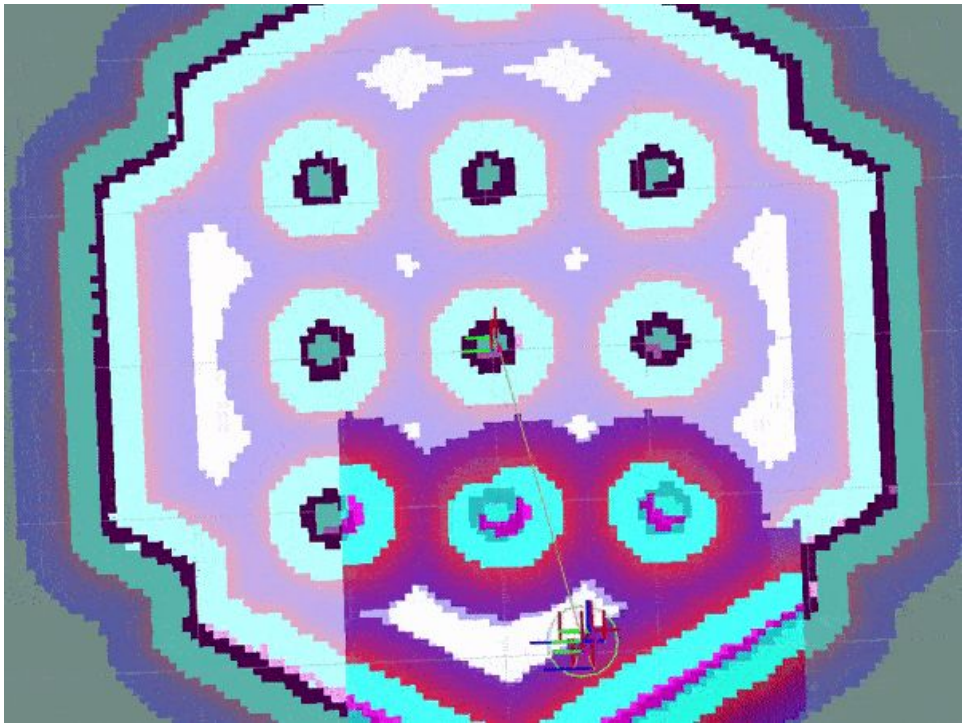


Controller server

- Also known as **local planner**, is the way we follow the globally computed path or complete a local task
- The controller will have access to a local environment representation to attempt to compute **feasible control efforts** (/cmd_vel) for the base to follow
- Many controller will project the robot forward in space and compute a locally feasible path at **each update iteration**
- Controllers can be written to:
  - Follow a path
  - Dock with a charging station using detectors in the odometric frame
  - Board an elevator
  - Interface with a tool

Controller server

- The general task in Nav2 for a controller is to compute a valid control effort to follow the global plan



Behavior server

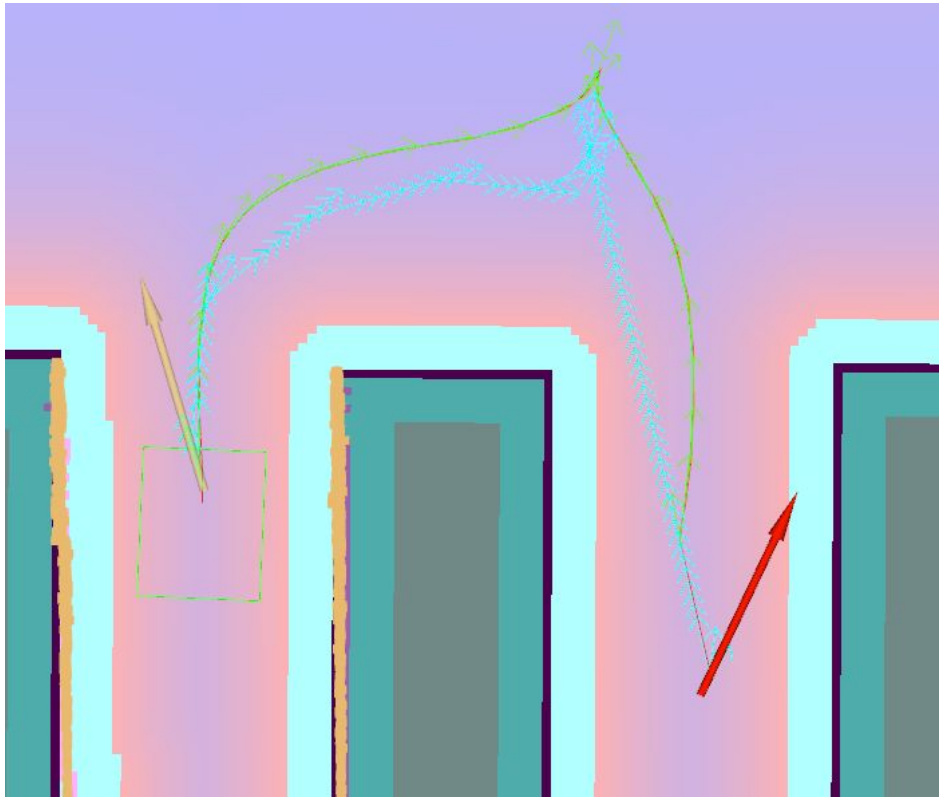
- Recovery behaviors are a mainstay of **fault-tolerant systems**
- The goal of recoveries are to **deal with unknown or failure conditions** of the system and autonomously handle them
- Examples may include faults in the perception system resulting in the environmental representation being full of fake obstacles, so the **clear costmap recovery** would then be triggered to allow the robot to move
- Another example would be if the robot was stuck due to dynamic obstacles or poor control, then, **backing up or spinning in place** allow the robot to move from a poor location into free space it may navigate successfully
- In the case of a total failure, a recovery may be implemented to call an operator’s attention for help, like a SMS

Smoother server

- As criteria for optimality of the path searched by a planner are usually reduced compared to reality, **additional path refinement** is often beneficial
- Smoothers have been introduced for this purpose, typically responsible for **reducing path raggedness** and **smoothing abrupt rotations**
- Also responsible for **increasing distance from obstacles and high-cost areas** as the smoothers have access to a global environmental representation
- It may be advantageous combining **different planners** with **different smoothers**, or use a separated smoother when a specific control over smoothing is required, e.g. smoothing only a specific part of the path

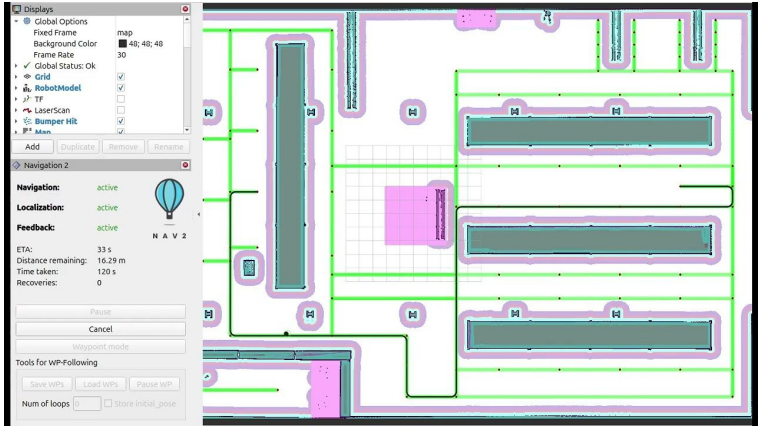
Smoother server

- The smoother receives a path (cyan) and return its improved version (green)



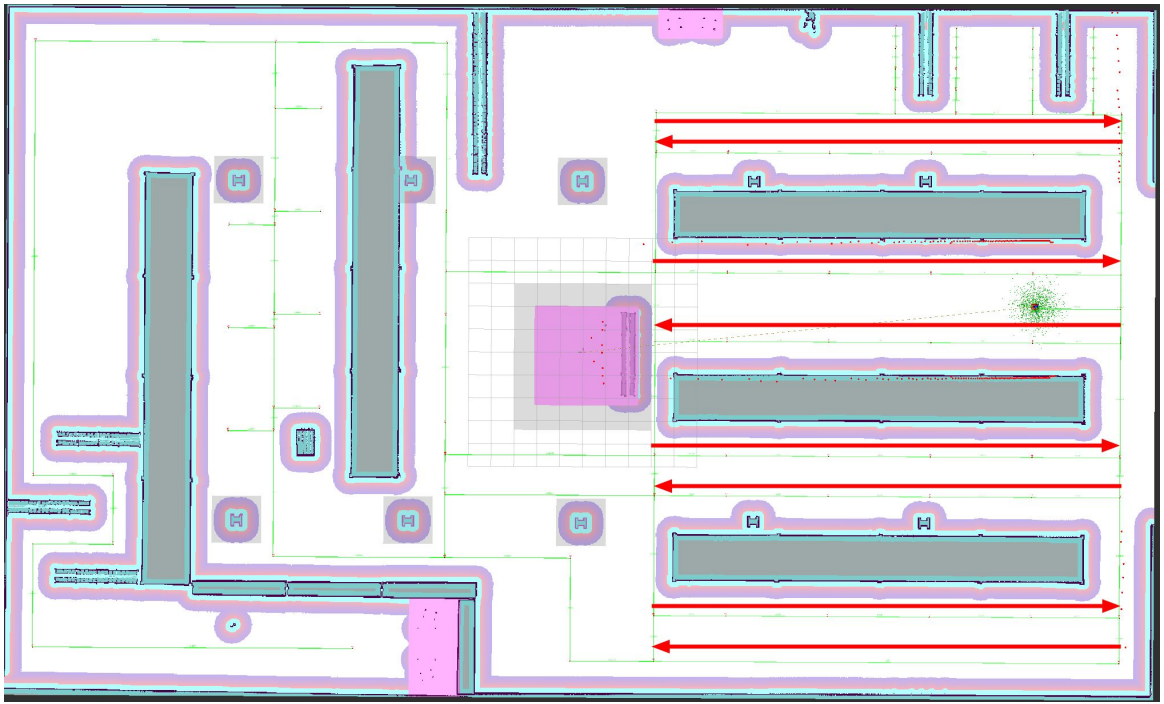
Route server

- A specialized planner that **computes a route using a navigation graph**, rather than the freespace costmap
- The route is computed as the optimal way from the start to the goal through the **set of nodes** and **directional edges** in the pre-defined navigation graph
- This navigation graph can be generated to represent lanes, areas the robot is allowed to navigate, a teach-and-repeat route, urban roadways



Route server

- It may be used to fully replace freespace planning when following a particular route closely or to augment the global planner with long-distance routing to a goal



Nav2 - words of wisdom

- Read the Navigation Concepts, it contains not only Nav2 concepts, but terminology, theoretical basis, great links for more concepts
- When implementing Nav2 in your project, make sure to read the Configuration Guide, it provides a process through which the user can adjust the tunable parameters (every single one) to obtain the best navigation performance
- If it’s time to fine tune your parameters (or even before), read the Tuning Guide, it provides really useful tips for the use of Nav2
- The General Tutorials provides several different applications, give it a try
- Don't be afraid to go back to the basics, to review your parameters, it may be really useful ;)

Thank You...

...for your attention

